NET-351: Python Final Project SDLC Guide

Goal: Build a real-world cybersecurity tool that solves a common issue cybersecurity analysts encounter.

Use the below Software Development Lifecycle guide to help you with the creation of your custom Python program/tool. This is a guide and not a direct "How-to" document.

Your final consists of four items:

1. Completed SDLC guide
2. Completed Python program code

SDLC Phases

Planning Phase

Objective: Define the scope, feasibility and initial requirements of the security tool

Tasks

a. Identify a one common security analyst challenge/task (e.g., log parsing, IOC enrichment, SIEM alert correlation).

A common challenge for security analysts is reviewing multiple Windows OS data sources such as running processes, netstat connections, and logs including System, Application, and Security Logs, to determine whether suspicious activity is present on the system. The goal of this program is to automate the collection and correlate this data from multiple tools.

b. Conduct feasibility study (Python's capabilities, API integrations, processing needs)

Yes, it can be done with Python; this program will utilize the subprocess module "subprocess.run()" function in the subprocess module

to run a bash script containing PowerShell commands that will collect running processes, the parent and child process IDs, network connections, and the application, security, and system logs from the system. Then, the script will output the data as txt files.

The Python program will then parse these collected text files and build process trees, and it will generate the SHA-256 hashes for each process. Then it will apply a risk score to the data, and if it is above a certain threshold of risk, then it will query the VirusTotal API. Then export these reports as CSVs.

It will also uses the modules os module for file paths and environment variables, subprocess module will being used for running the bash script, hashlib module will be used for hashing, pandas module will be used for the tables and CSV reports, time module will be for runtime tracking, and the vt module will be used for the integration of VirusTotal (if it is possible to integrate with in current timeframe). It will also output these reports into a folder it creates for it store the reports in.

c. Define core functionalities (e.g. automated threat lookup, log normalization, malware analysis)

This tool, when it is running it will collect data on the Windows system, match the activity in this data using the PID and PPID, and create parent-child process trees. Then this tool will generate SHA-256 hashes of executables. It will assign a risk score to executables based on suspicious behavior and then query VirusTotal when the risk score of an executable reaches a defined threshold. Finally, it will export these results into CSV reports.

d. Identify security considerations (e.g. data handling, authentication, access control)

Some of the potential security concerns for this program is the possible exposure of usernames and process paths, exposure of sensitive information in report files, there is a security risk with it running with elevated privileges, and there is a risk with the Virus Total API key being exposed. The free version of the VirusTotal API only allows for 4 requests permits and has a threshold of 10 processes at once if the program hits this threshold, then the API will block me.

Some ways to mitigate these issues are to submit only hashes to VirusTotal rather than full files, to validate script paths before the script is

executed, to use read-only PowerShell commands, the Virus Total API key should be stored in a secure repository rather than being hardcoded into the program. To prevent the program from reaching the threshold where VirusTotal blocks you, I will have to add a time delay between the requests.

e. Define initial project timeline and budget

Timeline of the project:

On May 1, I will complete my SDLC plan and the design of the program.

On May 3, I will write the Bash collection script and the python parsing and process tree generating.

On May 5, I will then add risk scoring and hashing. I will also try to implement VirusTotal into the program, and then I will test the program.

On May 6, I will do a final review of the program and then submit it.

Budget for the project:

There will be no cost required for this project if I use the public version of the VirusTotal API instead of the premium version.

Requirements Analysis Phase

Objective: Gather detailed functional and non-functional requirements

Tasks

a. Define functional requirements

   a. Input sources (SIEM logs, PCAPs, threat feeds, Sysmon logs, etc.)

      Windows data, such as running processes, PIDs and PPIDs, network connections, sha-256 hashes, system log, application log, and security log, will be collected by a bash script run by the python script.

   b. Output format (CSV, JSON, dashboard integration)

      The output data will be two CSV file, one for process trees and the other for the results from VirusTotal. These CSV files will be stored in a report folder created by the Python tool.

   c. Core features (e.g. log correlation, IOC enrichment, alert triage)

Core features to this tool are automated data collection, correlation of PID and PPID, generating a process tree, sha-256 hashing, risk scoring, creation of folders, creation of CSV reports, and VirusTotal lookups for suspicious executable.

b. Define non-functional requirements:
    a. Performance (processing speed, efficiency) What are the performance goals of the tool/program?

        This program should complete a normal scan in under five minutes on a Windows system
    b. Security (data encryption, role based access) Are there any security requirements at this time?

        The API keys and CSV reports must be protected in a secure environment to prevent a threat actor from exploiting these vulnerabilities caused by exposed API keys and CSV injection exploits.
    c. Scalability (handling increasing log volume) Does it need to scale?

        While the current version is for one Windows system, it can be scaled up to a larger environment in future versions.
c. Research external integrations (Are there any external systems you need to interface with? VirusTotal, Shodan, Elastic Stack etc.)

    External integration in this tool is with the VirusTotal API, used to check suspicious hashes and return the reputation results as a csv.

Design Phase

Objective: Define architecture, system components, and data flow

Tasks

a. Chose architecture (monolithic script, modular framework, API-based tool etc.)

    The tool architecture is a monolithic script that uses an external script for data collection.
b. Design data input pipeline:
    a. Creation/Gathering of data

        My tool will use the subprocess.run() function to launch a bash script that will execute PowerShell commands that gather logs, network data, and process information. This data will then return as text files for the rest of the Python script to parse.

c. Design data processing

When this program has completed the collection of the system data, Python will first have to normalize the data into a consistent format. Then it will match activity by PID, create process trees, calculate hashes, and assign a basic risk score. High scores will then trigger a VirusTotal lookup.

Implementation/Development Phase

Objective: Develop Python tool based on design specifications

Tasks

a. Setup development environment (Docker, virtual environment, GitHub repository, PyCharm etc.)

This tool will be developed in PyCharm using Python 3 and will be tested on a Windows 10 VM.

b. Write core features

The tool will run the bash automated data collection script with subprocess.run(), parse the returned information, correlation of PID and PPID, generating a process tree, SHA-256 hashing, risk scoring, VirusTotal lookups for suspicious executables, creation of folders, and creation of CSV reports.

c. Implement integrations (if required)

This tool will have to connect to the VirusTotal API for it hash lookups on high-risk process.

d. Security considerations
    a. Input validation & sanitization (Example)

    Inputs will be validated before parsing, and if they fails it will give an error message.

    b. Secure API key management (Example)

    The VirusTotal API key is to be stored using an environment variable to secure the API key.

    c. Logging and audit trail implementation (Example)

For logging of the program, it will keep track of scan time, successful runs, and errors

e. Code review & refactoring (Example)

For the Final code, I will be reviewing it before it is submitted for readability, efficiency, and for better organization. I will also use reusable function, if possible, when I am repeating code with the same logic on multiple functions.

f. Version control & CI/CD (GitHub) (Example)

For better source control and version tracking, I can use GitHub on this project.

Testing Phase

Objective: Identify and fix defects before deployment

Tasks

a. Unit testing (e.g. log ingest & parsing)

The function that will be tested individually will be the script execution function to see if subprocess.run() can successfully launch the bash script. Next will be the parsing function to confirm that the raw outputs from PowerShell are being converted by Python correctly.

I will then test the Hash function to ensure that it gives accurate SHA-256 hashes when using hashlib. The risk scoring function will also be tested with sample data to confirm that suspicious indicators, such as files being in the temp folder, are scored correctly.

The folder function will be tested to see if the report folder is being created, and I will also test the CSV generation function to make sure that pandas is exporting both report files into the report folder.

b. Integration testing

The tool will be tested to ensure that the bash script is correctly gives the data it collects to the Python script to confirm that the collected Windows process, network, and log data can be successfully parsed and that this data can be matched by PID.

I will also do integration testing to verify that the API key for VirusTotal is loaded correctly from the environment variable, as well as authenticate and perform hash lookups. There will also be error handling that will be

tested for invalid API keys, missing API keys, rate limits, and failed network connections.

As a final integration test will be done to confirm that all the components of this tool work together and can generate CSV reports correctly to the Reports folder.

c. Performance testing
I will test the tool's performance using the time module to make sure that it performs well on systems with many running processes.
d. Bug tracking & remediation
Bugs encountered during development will be documented and corrected before this project is submitted.

Deployment/Implementation

Objective: Deploy the tool for security analysts to use

Tasks

a. Package the tool
    a. Ready the tool for installation/deployment

        The tool will be in a folder that contains the main Python script, the Windows data collection bash file, a README.md, and a requirements.txt. When it is run, if there is not already a report folder, in the main tool folder then it will create the report folder with the two csv reports.
b. Deploy into production
c. This tool will be designed to run on Windows systems by security analysts.
d. Create user documentation
    a. How to install the tool
    b. How to use the tool
    c. Best practices

    There will be documentation on installation, how to run this tool, the setup of the API key, and how to read the reports that are generated.

e. Train security analysts on the tool

    Security analysts will be shown how this tool works and how the specific risk scores should be interpreted by the security analysts.

f. Monitor initial rollout and gather feedback

Any feedback gathered and data collected during the initial rollout will be used to improve future versions.

Maintenance & Support

Objective: Ensure long-term reliability and feature updates

Tasks:

a. Monitor performance and logs (identify errors, improve efficiency)

   In future versions, this program will monitor scan errors, slow execution times, and failed API lookups.

b. Patch security vulnerabilities (update Python or other dependencies)

   The Python libraries and dependencies will be updated on a scheduled on a regular basis to reduce risk.

c. Fix bugs reported by analysts

   The user-reported bugs will be reviewed and fixed in future releases.

d. Enhance features (e.g. new data ingestions)

   Some future improvements could be Sysmon support, JSON export, GUI support, and possibly a MITRE ATT&CK mapping.

e. Document updates and release new versions

   For all future updates, there will be version notes and updated documentation included.

Flowchart for Windows Baselining and Security Analyzing Tool

